CSC 2400-002: Design of Algorithms (Spring 2026)

Tennessee Tech University

Drill 20 Solutions

Topic: Dynamic Programming

ONLY FOR THE EYES OF STUDENTS IN **CSC 2400 SECTION 002, SPRING 2026, TENNESSEE TECH UNIVERSITY**. IF YOU ARE NOT PART OF THIS CLASS, PLEASE STOP READING IMMEDIATELY AND DELETE/DESTROY THIS FILE. A STUDENT SHARING THIS SOLUTION KEY WITH SOMEONE OUTSIDE THE CLASS/COURSE STAFF CONSTITUTES STUDENT ACADEMIC MISCONDUCT.

Question 1. Write the pseudocode of a dynamic programming algorithm that finds the Nth Fibonacci number using $\Theta(1)$ space.

Solution:

procedure CONSTSPACEFIB(N)
1: **if** $N = 1$ **then**:
2: **return** 1
3: $a \leftarrow 1,\ b \leftarrow 1, c \leftarrow 1$ ▷ initially $a = F[1]$, $b = F[2]$, and $c = b = F[2]$, the answer for $N = 2$
4: **for** $i = 3$ to N **do**: ▷ at this point, $a = F[i-2]$, $b = F[i-1]$, and $c = F[i-1]$
5: $c \leftarrow a + b$ ▷ c is overwritten by $F[i] = F[i-2] + F[i-1]$
6: $a \leftarrow b$ ▷ a is overwritten by $F[i-1]$
7: $b \leftarrow c$ ▷ b is overwritten by $F[i]$
8: **return** c

Question 2. The algorithm seen in class for the value version of the Coin-Row problem on input $C[1..n]$ maintains an array A of size $\Theta(n)$ in addition to the input array. Write an implementation (in pseudocode) of the same algorithm that uses $\Theta(1)$ space (in addition to the input array). Note that we just need the maximum amount collectable from the coins, not the set of coins that sum to it.

Solution: We see that to fill up ith entry $A[i]$ of the DP table A, we only need the knowledge of $A[i-2]$ and $A[i-1]$ in addition to the input array C. Thus, we can maintain only 3 entries of the table at a time and overwrite the previous entries that we no longer need with the new elements of A.

```
procedure CoinRow(C[1..n])
1: a ← 0, b ← C[1], c ← b ▷ initially a ← A[0], b ← A[1], and c = A[1], the answer for n = 1
2: for i = 2 to n do: ▷ at this point, a is set to A[i − 2] and b to A[i − 1]
3:     c ← max{b, C[i] + a} ▷ c is effectively set to A[i]
4:     a ← b ▷ a gets overwritten by A[i − 1]
5:     b ← c ▷ b gets overwritten by A[i]
6: return c
```

Question 3. Write the pseudocode of the dynamic programming algorithm seen in class for the Change Making problem: given a target amount n and coins of denomination $d_1, \ldots, d_k$, find the minimum number of coins needed to sum to n (assume that you have an unlimited supply of coins of each denomination).

Solution:

```
procedure MinChangeMaking(n, D[1..k]) ▷ Assume D[1] = 1 and D is sorted
1: A[0..n] ← [0, ∞, ..., ∞] ▷ 0 followed by n many ∞; takes care of base case A[0] = 0
2: for i = 1 to n do:
3:     j ← 1
4:     while j ≤ k and D[j] ≤ i do: ▷ we only need to check until the highest j with D[j] ≤ i
       since D is sorted
5:         A[i] ← min(A[i], A[i − D[j]] + 1)
6:         j ← j + 1
7: return A[n]
```

Question 4. Analyze the runtime of your implementation of the Change Making algorithm, where the basic operation is computing minimum of two numbers. Provide justification.

Solution: The outer loop runs for n values of i. The inner while loop runs for at most k iterations, where each iteration performs 1 minimum operation. Thus, the runtime is $\sum_{i=1}^{n} O(k) = O(nk)$.